

	UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERIA ESCUELA DE COMPUTACIÓN
CICLO 2	GUIA DE LABORATORIO #5 Programación Orientada a Objetos (POO404) Programación orientada a objetos (POO) Parte 3

I. OBJETIVOS

Que el estudiante:

- Se familiarice con el concepto de una interface (un tipo de clase abstracta) en Java
- Implemente el principio de POO denominado polimorfismo por medio de Interfaces.
- Diseñe en Java a clases con herencia multiple, implementando más de una interface.

II. INTRODUCCION

Polimorfismo y Abstraccion: diseño de Interfaces

Bajo los conceptos de diseño de programación orientada a objetos, una interfaz es una clase o plantilla abstracta que define un conjunto de métodos (y constantes) que otra clase (clase derivada) debe implementar.

Pero a diferencia de una clase abstracta, una interfaz no proporciona la lógica (cuerpo) de sus métodos, solamente a sus encabezados (Prototipos de métodos) y cuyas firmas (listado de parámetros) le indican "qué hará", no el "cómo se hará".

En otras palabras, una interfaz es un contrato que define qué métodos, propiedades o eventos deben implementar otras clases, pero sin dar detalles de cómo se hace.

Características de una interface:

- No tienen implementación, solo la "firma" de los métodos y definición de valores constantes.
- Sirven para garantizar que diferentes clases sigan un mismo contrato.
- Una clase puede implementar varias interfaces → herencia múltiple de comportamiento.
- Al igual que una clase abstracta, una interface no puede ser instanciada.
- Cuando una clase hereda de una interface, se afirma que *la clase esta implementando la interface*, lo que significa que debe sobrescribir (override) a todos y cada uno de los prototipo de métodos de la interface y brindarle una definición (cuerpo de metodo).

Las interfaces son fundamentales para la abstracción y el polimorfismo, obligando a las clases a tener un comportamiento específico.

En un Esquema de Herencia de Clases, una interfaz permite lograr mas flexibilidad y un tipo de herencia múltiple (una clase puede implementar varias interfaces).

Diseño de interfaces en Java

En Java, una interface se declara usando la palabra reservada `interface` y al crear una clase derivada de esta, se referencia con la palabra `implements`.

En Java, hay que tomar en cuenta los siguientes aspectos al definir e implementar interfaces:

- ✓ Se definen en archivos .java igual que a las Clases
- ✓ Los atributos (campos constantes) de la interfaz son por defecto `public static final`.
- ✓ Una interfaz no puede contener constructores, porque no se puede utilizar para crear objetos.
- ✓ Sus métodos de interfaz son por defecto `abstract public`.
- ✓ Al implementar una interfaz, la clase derivada debe anular (sobreescribir) a todos sus métodos.
- ✓ Para que una clase implemente varias interfaces, se deben redactar sepárandolas con comas (,)
- ✓ Una interface puede heredar de varias interfaces. Con las interfaces se permite la herencia múltiple.

A continuación se muestra un ejemplo del uso de interfaces en Java:

```
interface FirstInterface {
    //solo acepta prototipos (y su firma)
    void myMethod(); // interface method
}

interface SecondInterface {
    //una interface solo acepta campos fijos (constantes) y prototipos de metodos
    public final int constante = 4; //campo constante
    // metodo de esta interface (usa explicitamente public abstract)
    public abstract void myOtherMethod();
}

//Subclase DemoClass implementa a 2 interfaces distintas
class DemoClass implements FirstInterface, SecondInterface {
    //sobreescribe a cada prototipo (encabezado) de Los metodos de interfaces,
    //pero les brinda definicion (cuerpo)
    @Override
    public void myMethod() {
        System.out.println("Imprime un mensaje de texto..");
    }
    @Override
    public void myOtherMethod() {
        System.out.println("Imprime valor "+constante+" del campo constante");
    }
}
```

```

public class Main {
    //metodo de inicio de la aplicacion
    public static void main(String[] args) {
        //instancia objeto de subclase DemoClass
        DemoClass myObj = new DemoClass();
        myObj.myMethod();
        myObj.myOtherMethod();
    }
}

```

La etiqueta `@override` es opcional, ya que Java sobrescribe el prototipo heredado de una interface en la subclase que la implementa.

El resultado de la ejecución del ejemplo anterior sera el siguiente:

```

Run    Main x
C:\Users\ingen\.jdk\azul-17.0.8.1\bin\java.exe "-javaagent:C:\Program Files\
Imprime un mensaje de texto..
Imprime valor 4 del campo constante
Process finished with exit code 0

```

Simbologia para representar Interfaces en UML

Bajo un diagrama de clases UML, una interface puede representarse de 2 maneras diferentes. Por ej.

La interface (FirstInterface) mostrada en el ejemplo de la sección anterior de esta introducción teórica, puede representarse así:

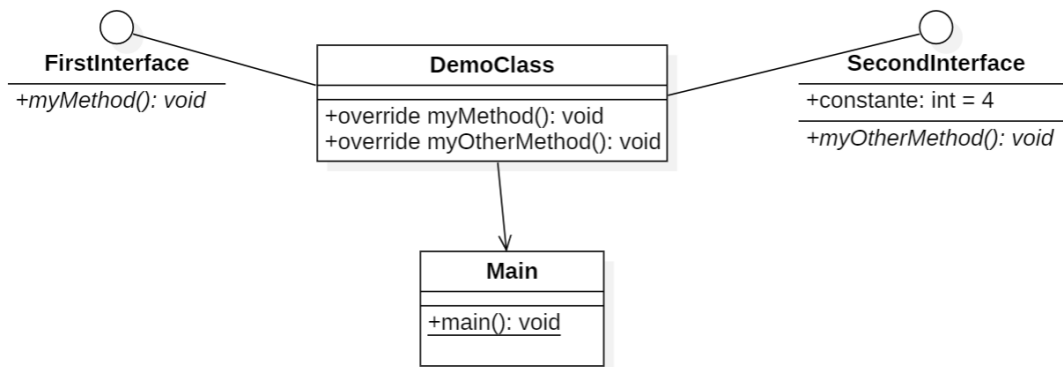


Si se compara el símbolo de interface con el simbolo de una clase estándar, se indica un *calificativo* (llamado también *estereotipo*) llamado **Interface** y sus prototipos de métodos se resaltan como públicos y abstractos.

La 2da nomenclatura indica que la interface es requerida. En esta materia se utilizará la 2da notación, la cual esta disponible en el software de StarUML.

Finalmente, para definir la implementación de una interface en una clase, en el primer formato se usa una flecha denominada “Realizacion de interface”:

Observe el diagrama UML del ejemplo descrito en esta introducción:



Operador instanceof

instanceof es un operador binario que se utiliza para comprobar si un objeto pertenece a un tipo/clase, una subclase o interfaz determinado. El resultado de la operación es lógico (**true** o **false**). También se conoce como *operador de comparación de tipos*, ya que compara una instancia con el tipo/clase.

Se recomienda utilizar la comprobación instanceof antes de convertir un objeto desconocido. Esto ayuda a evitar una excepción del tipo `ClassCastException` en tiempo de ejecución, un error que resulta de convertir un objeto en una clase diferente.

Cómo funciona el operador instanceof

El operador instanceof funciona según el principio de la relación **es-un**. El concepto de una relación **es-un** se basa en la herencia de clases o en la implementación de interfaces.

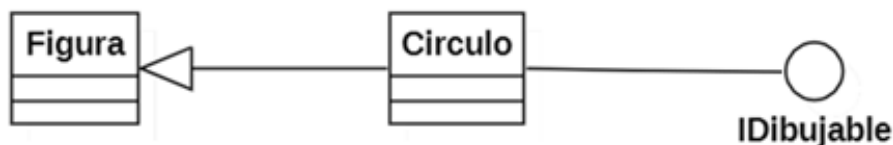
Al ejecutar el operador instanceof, se compara una variable de referencia con una clase específica también indicada. El operador se limita a determinar si hay coincidencia o no, sin incluir más información sobre la naturaleza del objeto o de la clase. La sintaxis básica del operador instanceof es:

(objeto) instanceof (tipo o clase)

Si (objeto) es una instancia de (clase, subclase o interface), retorna true, de lo contrario, retorna false.

Ejemplo:

Observe el siguiente diagrama UML, en el cual se define a una clase *Circulo*, la cual se deriva/hereda de la clase *Figura* e implementa a la interface *IDibujable*,



Con el esquema de clases anterior, se proporciona el siguiente código que demuestra el uso del operador instanceof:

```

Circulo circulo1= new Circulo();
IDibujable circulo2 = new Circulo();
Figura figura1= new Figura();
if(circulo1 instanceof Circulo)
    System.out.println("A. circulo1 es objeto de la clase Circulo");
else
    System.out.println("A. circulo1 no es objeto de la clase Circulo");

if(circulo2 instanceof Figura)
    System.out.println("B. circulo2 es objeto de la clase Figura");
else
    System.out.println("B. circulo2 no es objeto de la clase Figura");

if(circulo2 instanceof IDibujable)
    System.out.println("C. circulo2 implementa a interface IDibujable");
else
    System.out.println("C. circulo2 no implementa a interface IDibujable");

```

Al ejecutar la aplicación, los resultados serán los siguientes:

```

A. circulo1 es objeto de la clase Circulo
B. circulo2 es objeto de la clase Figura
C. circulo2 implementa a interface IDibujable

```

Observe que el operador `instanceof` permite definir que instancia `circulo2` es una instancia de subclase *Circulo* (derivada de clase *Figura*) y también permite alojar instancia que implementan a miembros de interface *IDibujable*.

Este operador también funciona con la clase `Object`, la cual es la clase base del resto de clases en Java:

```

if(circulo1 instanceof Circulo)
    System.out.println("D. circulo1 es objeto de la clase generica Object");
else
    System.out.println("D. circulo1 no es objeto de la clase generica Object");

```

Resultado al ejecutar esta prueba:

```

D. circulo1 es objeto de la clase generica Object

```

Las superclases no son una instancia de una clase inferior

Mientras que, en la jerarquía de clases, los objetos de las subclases también pertenecen a las clases superiores, este principio no funciona a la inversa. Esto se puede ver en el siguiente ejemplo:

```

if(figura1 instanceof Circulo)
    System.out.println("E. figura1 es objeto de la subclase Circulo");
else
    System.out.println("E. figura1 no es objeto de la subclase Circulo");

```

Resultado al ejecutar esta prueba:

E. figura1 no es objeto de la subclase Circulo

¿Qué es Java casting?

En el proceso de gestionar tipos de datos o convertirlos en otros tipos, es necesario conocer acerca del Java casting.

Para iniciar, hay que recordar que los tipos de datos en Java están divididos en dos categorías principales:

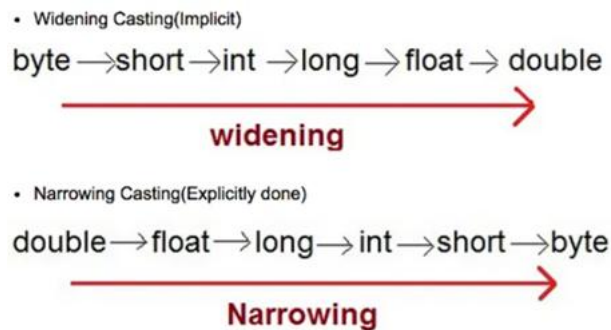
- a) datos primitivos
- b) datos referenciados (objetos)

Los datos primitivos de java son 8 tipos: boolean, byte, char, int, short, long, float y double

Los datos referenciados abarcan cualquier clase u objeto. **El Java casting es, básicamente, un proceso que consiste en convertir un valor de un tipo de dato a otro.**

Tipos de Java casting

Existen dos tipos de Java casting: el widening casting (conversión automática) y el narrowing casting (conversión explícita, manual).



Widening casting (conversión automática o implícita)

El widening casting es un proceso que **se realiza de manera automática cuando se convierte un tipo de dato más pequeño (por ej. int) en uno más grande (a por ej. double)**. Java realiza automáticamente esta conversión, ejemplo:

```
int entero = 9;
double decimal = entero; // Casting automático: int a double
System.out.println(entero); // Imprime 9
System.out.println(decimal); // Imprime 9.0
```

Narrowing casting (conversión explícita, manual)

Contrario al metodo anterior de conversion, el narrowing casting **sucede cuando se tiene que convertir un tipo de dato más grande a uno más pequeño y tiene que hacerse de forma manual**. En esta conversión, se puede perder información importante. Un ejemplo de narrowing casting es el siguiente:

```
double decimal = 3.78;
int entero = (int) decimal; // Casting manual: double a int
System.out.println(decimal); // Imprime 3.78
System.out.println(entero); // Imprime 3
```

En este caso, se convirtió manualmente un valor double a un int. El valor decimal 3.78 se ha truncado a 3, lo que demuestra la pérdida de información en un narrowing casting.

Java casting en objetos

El proceso de java casting se puede ejecutar con datos primitivos y también con datos referenciados (los objetos).

Los objetos de una clase se pueden convertir en objetos de otra clase, solamente si ambas clases están relacionadas entre sí a través de la propiedad de herencia, es decir, existe una clase padre y la otra es una clase hija.

El caso más sencillo de casting es cuando, debido a que **todas las clases en Java son subclases de la clase genérica Object**, muchas veces se necesita hacer un java casting para poder convertir un objeto genérico a otro de su tipo específico. Veamos un ejemplo de cómo hacerlo:

```
//Colección Map que guarda objetos de clase generica Object
Map<String, Object> dispositivos = new HashMap<>();

//almacena objeto de clase Televisor como un Object
dispositivos.put("televisor", new Televisor());

//casting de clase genérica Object a clase (subclase) especifica Televisor
Televisor tv = (Televisor) dispositivos.get("televisor");
tv.encender(); //invoca miembro especifico de clase Televisor
```

Otro ejemplo:

A continuación, la clase padre es **Animal** y la clase **Dog** es la clase hija.

```
class Animal{
    protected String name;
    public Animal(String name){
        this.name = name;
    }
    public void animalInfo() {
        System.out.println("clase Animal class (info): ");
        System.out.println("** Name: "+this.name);
    }
}

class Dog extends Animal {
    public String color;

    public Dog(String name, String color){
        super(name);
        this.color = color;
    }
}
```

```

public void dogInfo() {
    System.out.println("clase Dog (info): ");
    System.out.println("Name: "+this.name +
        ", Color: "+this.color);
}

public static void main(String[] args) {
    Animal animal = new Dog("Odie", "Yellow");
    //Casting explicito de objeto, de clase Animal a clase Dog
    Dog odie = (Dog) animal;
    odie.animalInfo();
    odie.dogInfo();
}
}

```

El resultado sera el siguiente:

```

class Animal class (info):
** Name: Odie
class Dog (info):
**Name: Odie, Color: Yellow

```

Ampliación de la conversión de tipos con objetos (Upcasting)

Caso contrario ocurre cuando se requiere convertir un objeto de una clase hija a otro de una clase padre.

En este tipo de conversión (implícita), el objeto de la clase padre contendrá a solamente los miembros del objeto de la subclase.

Ejemplo:

Si se retoma el ultimo ejemplo, de las clases (Animal y Dog), un ejemplo de conversión implícita sera el siguiente:

```

public static void main(String[] args) {
    Dog Snoopy = new Dog("Snoopy","White");
    Animal animal = new Animal("Astro");
    //Convertir implicitamente Object de Dog a Animal
    animal = Snoopy;
    animal.animalInfo();
}

```

Su ejecución genera el siguiente resultado. Observar que objeto animal solo puede usar miembros de la subclase:

```

class Animal class (info):
** Name: Snoopy

```

III. PROCEDIMIENTO

PARTE 1: implementación de una interface

1. Desde el IDE IntelliJ-IDEA, cree un nuevo proyecto java con el nombre **POOGuia5proc**.
2. En la carpeta **src**, cree a 3 paquetes llamados **ejemplo1**, **ejemplo2** y **ejemplo3**.
3. En el paquete **ejemplo1** cree a una clase con el nombre **Aplicacion1**.

Luego, en la definición de esta clase redacte al método estático de inicio **main**.

4. Dentro del mismo paquete **ejemplo1**, creara a una interface llamado **IPago**.

Para realizarlo, haga clic secundario sobre el paquete y seleccione *new -> java class*.

Pero esta vez seleccione la opción *Interface* y asigne el nombre **IPago**, tal como se muestra a continuación:

5. Defina el siguiente código para la nueva interface **IPago**:

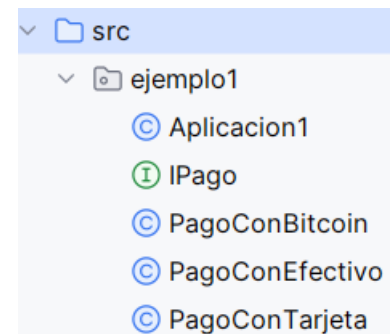
```
package ejemplo1;

public interface IPago {
    //prototipos de metodos y su firma (lista de parametros)
    String ProcesarPago(float monto);
}
```

6. Ahora que la interfaz está creada, creara a 3 clases que implementen a esta interfaz.

Para ello, dentro del paquete **ejemplo1**, cree a las clases **PagoConTarjeta**, **PagoConBitcoin** y **PagoConEfectivo**.

Si observa la ventana de la estructura de su proyecto, el paquete **ejemplo1** tendra una interface y 4 clases:



7. Abra el contenido del archivo de la clase **PagoConTarjeta**. Utilice la palabra **implements** en el encabezado de la clase, para que esta implemente a la interface **IPago**:

```
public class PagoConTarjeta implements IPago
```

Con el cambio anterior, el IDE informa que hay errores. Coloque el cursor del ratón sobre el encabezado de la clase y lea la descripción del error informado.

8. Para solucionar el error, el IDE le reclama definir el cuerpo de **cada metodo heredado de la interface IPago**. Por lo tanto, en el interior de la clase **PagoConTarjeta**, redacte la siguiente definici3n para el metodo **ProcesarPago**:

```
package ejemplo1;

public class PagoConTarjeta implements IPago {
    @Override
    public String ProcesarPago(float monto) {
        String resultado =
            String.format("Pago con Tarjeta del monto recibido es $ %.2f",
                           monto);
        return resultado;
    }
}
```

9. Abra la clase **PagoConBitcoins** y repita el par de pasos anteriores para modificar a esta clase, haciendo que implemente de la interface **IPago** y redactando la siguiente definici3n para su metodo heredado **ProcesarPago**:

```
public String ProcesarPago(float monto) {
    String resultado =
        String.format("Pago con Bitcoins del monto recibido es $ %.2f", monto);
    return resultado;
}
```

10. Abra a la clase **PagoConEfectivo** y modifiquela de manera similar al cambio realizado con las 2 clases anteriores que implementan la interface **IPago**. El mensaje por retornar ser4:

"Pago en Efectivo del monto recibido es \$ %.2f", monto

11. Retorne a la clase **Aplicacion1** y digite la siguiente definici3n para su metodo de inicio **main**:

```
public static void main(String[] args) {
    //objeto que almacenara solamente a instancias de clases derivadas de interface
    IPago
    IPago metodopago = null;

    Scanner sc = new Scanner(System.in);
    float monto; //valor de monto a pagar
    int tipopago; //eleccion de metodo para pagar monto
    String resultado;

    System.out.println("Tienda de conveniencia\n");
    System.out.print("Ingrese valor del monto a realizar: $ ");
    monto = sc.nextFloat();
    System.out.println("\nMetodos de Pago:");
    System.out.println("\t1. Tarjeta de credito\n\t2. Bitcoins\n\t3. Efectivo");
    System.out.println("Seleccione numero con metodo de pago del monto ingresado: ");
    tipopago = sc.nextInt();
}
```

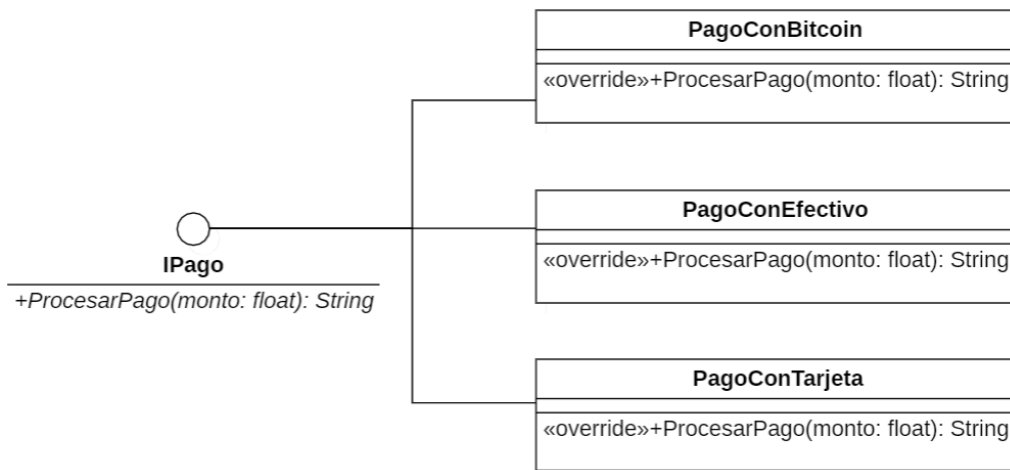
```

switch (tipopago) {
    case 1:
        metodopago = new PagoConTarjeta();
        break;
    case 2:
        metodopago = new PagoConBitcoin();
        break;
    case 3:
        metodopago = new PagoConEfectivo();
        break;
} //fin switch tipo pago
//ejecuta el pago con metodo elegido
if(metodopago == null)
    System.out.println("ERROR: No eligio un metodo de pago valido");
else{
    resultado = metodopago.ProcesarPago(monto);
    System.out.println(resultado);
}
}

```

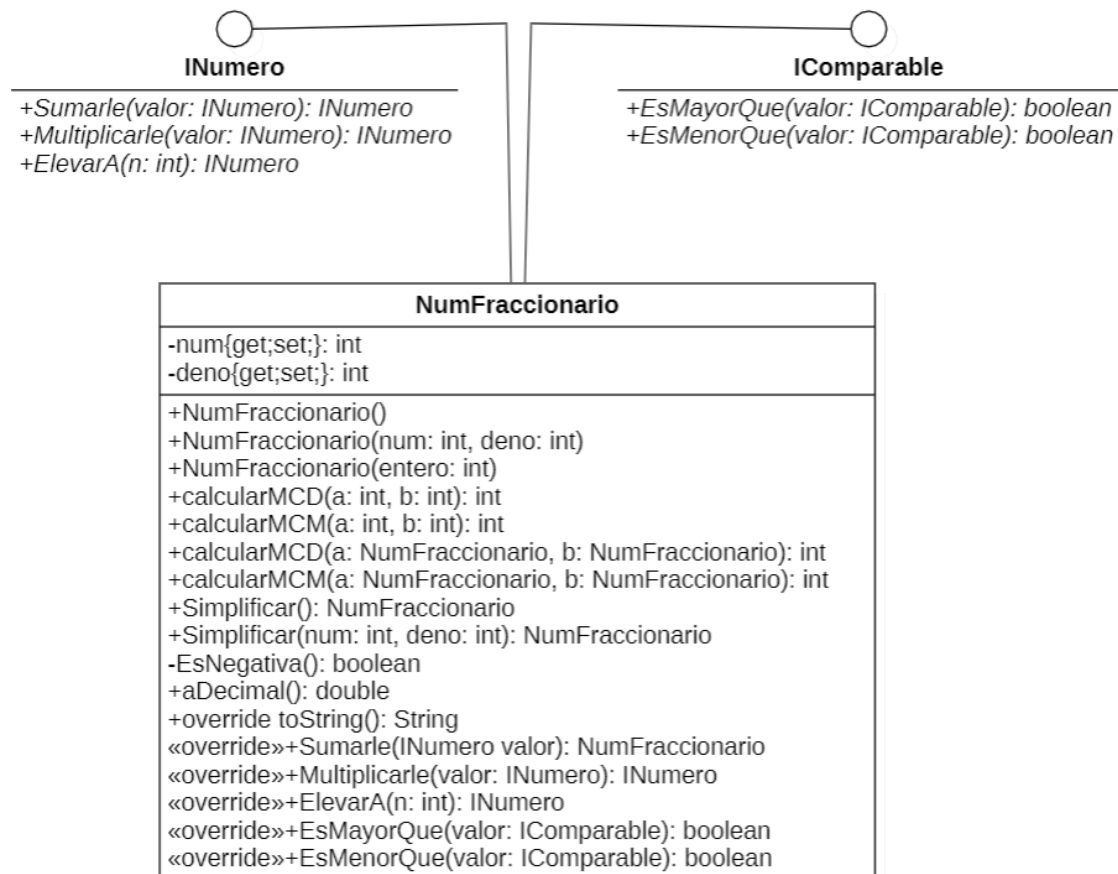
Del código anterior, observe que el objeto *metodopago* nunca instancia a la interface **IPago**. En cambio, si puede referenciar a objetos instanciados de solamente las subclases que implementan a la interface **IPago**.

- 12.Proceda a ejecutar a la clase Aplicacion1. Ingrese los datos solicitados y de acuerdo con la selección del metodo de pago, usted determine cual subclase fue utilizada.
- 13.Repita la prueba y seleccione un numero incorrecto de metodo de pago y verifique el error devuelto.
- 14.A continuación, se le muestra el diagrama UML de las clases e interfaces definidas en el paquete **ejemplo1**. Observe que la interface **IPago** se realiza en 3 clases distintas:



PARTE 2: Herencia multiple con 2 interfaces

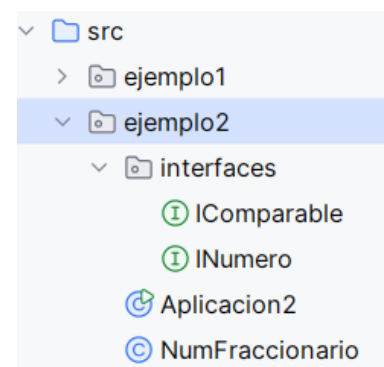
15. En este 2do Ejemplo implementara el siguiente esquema UML de clases:



16. Desde el diseñador del proyecto, en la carpeta *src* cree otro paquete, con el nombre **ejemplo2**.

Luego, en el interior de este paquete, el siguiente esquema de clases e interfaces:

Importante: Cuide de diferenciar cuales elementos son *clases* y cuales son *interfaces*, así como *paquetes*.



17. Ingrese a la definición de la interface **IComparable** y digite el siguiente código:

```
package ejemplo2.interfaces;

public interface IComparable {
    boolean EsMayorQue(IComparable valor);
    //boolean EsMenorQue(IComparable valor);
}
```

18. De manera similar, ingrese al archivo de la interface **INumero** y digite el siguiente código:

```
package ejemplo2.interfaces;

public interface INumero {
    INumero Sumarle(INumero valor);
    //INumero Multiplicarle(INumero valor);
    //INumero Dividir(INumero valor);
    //INumero ElevarA(int n);
}
```

19. Abra la definición de la clase **NumFraccionario**. Tal como se indica en el diagrama UML, esta clase implementará a las 2 interfaces. Para ello, modifique el encabezado de la siguiente manera:

```
public class NumFraccionario implements INumero, IComparable
```

Importante:

- ✓ Observe que el listado de interfaces se separa con comas (,)
- ✓ Además, Java reclama la implementación de *los prototipos de métodos* definidos en ambas interfaces.

20. Para solucionar el error de Java que ordena implementar a los prototipos de métodos de ambas interfaces, temporalmente escriba los siguientes métodos:

```
//Sobreescripciones de prototipos indicados en ambas interfaces

@Override
public NumFraccionario Sumarle(INumero valor) {
    return null;
}

@Override
public boolean EsMayorQue(IComparable valor) {
    return false; //fraccion local es menor que fraccion recibida
}
```

Importante:

- El cuerpo de estos métodos implementados es solo temporal, serán modificados en el resto de este procedimiento.
- Cada método utiliza la etiqueta **@Override**, pero esta puede ser ignorada, ya que Java reemplazará el *prototipo del método* de la interface por su *definición* en la clase que implementa a este método.

21. Coloque el cursor justo luego de la llave de apertura de la clase **NumFraccionario** y presione varias veces a la Tecla Enter. Esto permitirá insertar miembros a esta clase, ubicándolos antes de las definiciones de los métodos insertados en el paso anterior.

22. En las líneas vacías al inicio de esta clase **NumFraccionario**, declare a los campos indicados en el UML para esta clase. Cuide que sean de acceso privado.

23. Proceda a encapsular los campos declarados en el paso anterior.

Luego, modifique el método *setter* del campo **deno**, con esta definición:

```
public void setDeno(int deno) {  
    // para evitar una fraccion con denominador cero, ej: 8/0, -4/0, etc.  
    if(deno!=0)  
        this.deno = deno;  
}
```

24. Digite las siguientes definiciones del método constructor de esta clase:

```
public NumFraccionario(){  
    //asigna entero 0, pero en formato de una fraccion (0/1)  
    setNum(0);  
    setDeno(1);  
}  
public NumFraccionario(int num, int deno) { // Sobrecarga 1  
    setDeno(1); //evitar fraccion con denominador 0  
  
    setNum(num);  
    setDeno(deno);  
}  
public NumFraccionario(int entero){ // Sobrecarga 2  
    //para almacenar un entero como una fraccion, ej: 3 se expresara como 3/1  
    setNum(entero);  
    setDeno(1);  
}
```

25. Ahora digite el siguiente conjunto de métodos:

```

// reemplaza metodo toString heredado de clase Object,
// para retornar como String a vista de la fraccion almacenada
@Override
public String toString() {
    if(deno==1)
        return String.valueOf(num);
    return num + "/" + deno;
}

//Operaciones propias sobre solamente fracciones (a/b)
// Metodo para calcular el Máximo Común Divisor (Algoritmo de Euclides)
public int calcularMCD(int a, int b) {
    int temp;
    while (b != 0) {
        temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// Metodo para calcular el MCM usando el MCD
public int calcularMCM(int a, int b) {
    int mcm;
    if (a == 0 || b == 0) {
        return 0; // EL MCM con 0 es 0
    }
    mcm = (a * b) / calcularMCD(a, b);
    return mcm;
}

public int calcularMCM(NumFraccionario a, NumFraccionario b){ //(sobrecarga)
    //extrae denominadores para invocar a sobrecarga que recibe solo denominadores
    return calcularMCM(a.getDeno(),b.getDeno());
}

public int calcularMCD(NumFraccionario a, NumFraccionario b){ //(sobrecarga)
    //extrae denominadores para invocar a sobrecarga que recibe solo denominadores
    return calcularMCD(a.getDeno(),b.getDeno());
}

```

```

// simplificar valor de fraccion interna de esta instancia
public NumFraccionario Simplificar(){
    int mcd = calcularMCD(getNum(), getDeno());
    setNum(getNum()/mcd);
    setDeno(getDeno()/mcd);
    //de manera opcional, retorna una nueva instancia de fraccion simplificada
    return new NumFraccionario(getNum(),getDeno());
}

//(sobrecarga)
// simplifica fraccion recibida con numerador y denominador recibidos
public NumFraccionario Simplificar(int num, int deno){
    NumFraccionario r = new NumFraccionario();
    int mcd = calcularMCD(num, deno);

    r.setNum(num/mcd);
    r.setDeno(deno/mcd);
    return r;
}
private boolean EsNegativa(){
    //retorna true si numerador de fraccion interna es negativa
    if(this.getNum()<0) return true;
    return false;
}

public double aDecimal(){
    //retorna valor de cociente de dividir numerador y denominador
    return (double)this.getNum()/((double)this.getDeno());
}

```

26. Ahora comprueba a los métodos digitados en el paso anterior.

Retorne a la clase **Aplicacion2** y cree la definición vacía del método estático de inicio **main**.

27. Digite el siguiente código en el método **main** definido en el paso anterior:

```

//instancias de clase NumFraccionario
NumFraccionario f0, f1, f2, f3, f4;
NumFraccionario r1,r2,r3;
//para almacenar calculos
int mcm, mcd;
System.out.println("Operaciones con Fracciones");
//inicializa instancias
f0 = new NumFraccionario();
f1 = new NumFraccionario(2,5); // 2/5
f2 = new NumFraccionario(3,8); // 3/8
f3 = new NumFraccionario(-168,210); // -168/210
f4 = new NumFraccionario(3); // 3/1
System.out.format("Las fracciones para las pruebas seran %s, %s, %s, %s y %s\n\n",
    f0, f1,f2,f3,f4);

```

```

mcm = f0.calcularMCM(630,1050);
mcd = f0.calcularMCD(630,1050);
System.out.format("El mcm de 630 y 1050 es %d, el mcd es %d\n",mcm,mcd);

mcm = f0.calcularMCM(f1,f2);
mcd = f0.calcularMCD(f1,f2);
System.out.format("El mcm de los denominadores de %s y %s es %d, el mcd es %d\n",
    f1,f2,mcm,mcd);
System.out.format("La fraccion equivalente y simplificada de %s ",f3);
r1 = f3.Simplificar();
System.out.format("es %s\n",r1);

System.out.format("La fraccion equivalente y simplificada de 300/270 es %s\n",
    f0.Simplificar(300,270));

f0 = new NumFraccionario(-23,125); //fraccion -23/125
System.out.format("El valor decimal de la fraccion %s es %.4f\n",
    f0, f0.aDecimal());

```

28.Proceda a ejecutar a la clase Aplicacion2. El resultado sera el siguiente:

```

Run  Aplicacion2 x
C:\Users\ingen\.jdk\azul-17.0.8.1\bin\java.exe "-javaagent:C:\P
Operaciones con Fracciones
Las fracciones para las pruebas seran 0, 2/5, 3/8, -168/210 y 3

El mcm de 630 y 1050 es 3150, el mcd es 210
El mcm de los denominadores de 2/5 y 3/8 es 40, el mcd es 1
La fraccion equivalente y simplificada de -168/210 es -4/5
La fraccion equivalente y simplificada de 300/270 es 10/9
El valor decimal de la fraccion -23/125 es -0,1840

Process finished with exit code 0

```

Haga el calculo manual del MCM y del MCD de los pares de valores (630 y 1050) y de los denominadores (5 y 8). Verifique que coinciden con los valores retornados por los métodos de la clase **NumFraccionario**.

Y verifique el resto de los resultados.

29.Ahora, en la clase **NumFraccionario** digitara la definición correcta para cada metodo implementado de las interfaces. Ubique el cursor dentro del metodo **Sumarle**, borre la linea *return* actual y digite el siguiente código:

```
//define fraccion con valor cero (0/1), en caso que no pueda hacer la operacion
NumFraccionario s = new NumFraccionario();
//prueba si instancia recibida es de este mismo tipo de clase NumFraccionario
if (valor instanceof NumFraccionario) {
    NumFraccionario a, b;
    a = new NumFraccionario(getNum(), getDeno()); //valor de fraccion interna
    b = (NumFraccionario) valor; //convierte parametro a objeto de clase NumFraccionario
    //Calcula suma de fraccion interna de esta instancia con objeto recibido en parametros
    //y actualiza valor de fraccion interna con este resultado
    int mcm = calcularMCM(a.getDeno(), b.getDeno());

    s.setNum(mcm/getDeno()*getNum()+ mcm/b.getDeno()*b.getNum());
    s.setDeno(mcm);
    //simplifica fraccion obtenida
    s = Simplificar(s.getNum(), s.getDeno());
}
return s;
```

30. De manera similar al paso anterior, borre el contenido actual del metodo **EsMayorQue** y a cambio, digite el siguiente código:

```
if(valor instanceof NumFraccionario){
    //compara fraccion interna con fraccion recibida en objeto
    NumFraccionario p1,p2;

    p1 = new NumFraccionario(getNum(), getDeno());
    p2 = (NumFraccionario) valor;

    if(!p1.EsNegativa() && !p2.EsNegativa())//Si ambas son positivas
    {
        if(p1.aDecimal()>p2.aDecimal()) return true;
    }
    else
        if(p1.EsNegativa() && p2.EsNegativa()) //si ambas son negativas
        {
            //cambia sentido de prueba
            if(p2.aDecimal()>p1.aDecimal()) return true;
        }
        else
            //tienen signos distintos, si fraccion interna tiene signo positivo, sera mayor
            if(!this.EsNegativa()) return true;
    }
    return false; //fraccion local es menor que fraccion recibida
```

31. Retorne a la clase **Aplicacion2** e inserte las siguientes líneas de código:

```
r2 = f1.Sumarle(f2); //calcula suma de 3er numero con 4to numero
System.out.format("\nLa suma de %s con %s es %s\n", f1,f2,r2);
```

```
//suma un entero con la fraccion
f0 = f1.Sumarle(f4);
System.out.format("Suma de %s y %s es %s\n", f1,f4,f0);

if(f1.EsMayorQue(f4)) //compara fraccion f1 con f4
    System.out.format("La fraccion * %s es mayor que %s\n",f1,f4);
else
    System.out.format("La fraccion %s es mayor que %s\n",f4,f1);
```

32. Vuelva a ejecutar su aplicación. En la imagen se resaltan los cálculos de la implementación de los métodos de la interface:

```
Run  Aplicacion2 x
El mcm de los denominadores de 2/5 y 3/8 es 40, el mcd es 1
La fraccion equivalente y simplificada de -168/210 es -4/5
La fraccion equivalente y simplificada de 300/270 es 10/9
El valor decimal de la fraccion -23/125 es -0,1840
La suma de 2/5 con 3/8 es 31/40
Suma de 2/5 y 3 es 17/5
La fraccion 3 es mayor que 2/5
Process finished with exit code 0
```

33. Tome de modelo al método **Sumarle** y defina el siguiente prototipo en la interface **INumero**:

INumero **Dividir**(**INumero** valor);

Luego, en la clase **NumFraccionario** define su implementación correspondiente. Tome en cuenta los siguientes ejemplos de ejecución del valor que debería retornar este método:

*Asumiendo que el valor actual almacenado en la instancia fuera **2/3**, los resultados deberían ser:*

Al invocar al metodo con	Resultado devuelto
Dividir(new NFraccionario(4,9))	$\frac{2/3}{4/9} = \frac{3}{2}$
Dividir(new NFraccionario(-3,7))	$\frac{2/3}{-3/7} = \frac{-14}{9}$

Dividir(new NFraccionario(-28,16))	$\frac{2/3}{-28/16} = \frac{-8}{21}$
Dividir(new NFraccionario(16))	$\frac{2/3}{-16/1} = \frac{-1}{24}$

Haga la demostración con los ejemplos indicado en esta tabla.

34. Tome de modelo al metodo **Sumarle** para definir el siguiente prototipo en la interface **INumero**:

INumero **ElevarA**(int n);

Luego, en la clase NumFraccionario defina el metodo **ElevarA** implementado de la interface INumero.

Tome en cuenta los siguientes ejemplos sobre el calculo de potencias de fracciones:

$$\begin{aligned} \left(\frac{1}{3}\right)^2 &= \frac{1}{9} & \cdot & & (3)^{-4} &= \frac{1}{81} \\ \left(\frac{1}{2}\right)^{-3} &= 8 & & & \left(\frac{-6}{10}\right)^{-3} &= \frac{-125}{27} \\ \left(\frac{3}{2}\right)^3 &= \frac{27}{8} & & & \left(\frac{12}{16}\right)^3 &= \frac{27}{64} \end{aligned}$$

Modifique la clase Aplicacion2 para incluir a 3 demostraciones del uso del metodo ElevarA.

35. Guarde su proyecto y llame a su instructor para mostrarle los resultados de la ejecución.

IV. EJERCICIOS COMPLEMENTARIOS

25% Ejercicio 1:

En la clase **NumFraccionario**, modifique la sobreescritura del metodo **toString** para que, al identificar que su fracción interna es una *fracción impropia*, la retorne en formato *de fracción mixta*. Luego, demuestre su uso con 3 fracciones, por ej.:

$$52/14 = 3(5/7) \quad -280/84 = -3(1/3) \quad -525/140 = -3(3/4)$$

25% Ejercicio 2:

En la clase **Aplicacion2**, incluya un metodo que reciba una colección con objetos de la clase **NumFraccionarios**, para luego, imprimirlos en la pantalla y finalmente, mostrar el valor de la suma de todos los valores recibidos. Para que haga su tarea, este metodo debe recibir al menos una fracción, de lo contrario, se debe mostrar el mensaje:

Datos recibidos son incompletos, fin de ejecución del metodo

Finalmente, haga lo necesario para demostrar el funcionamiento de este metodo solicitado.

50% Ejercicio 3:

Cree una clase que implemente solamente a la interface **INumero** usada en la Parte 2 de este procedimiento.

Esta nueva clase permitirá abstraer el concepto de numero complejo ($a + b.i$), cuyo valor consta de una parte entera (a) y de un numero imaginario ($b.i$).

Implemente los miembros necesarios para almacenar un numero complejo, que incluya el encapsulamiento de sus campos y la definición de los métodos de los prototipos de métodos incluidos en la interface a implementar.

Modifique el metodo **toString** para imprimir apropiadamente el valor complejo, observe algunos ejemplos:

<i>Valor num. complejo almacenado</i>	<i>Valor a imprimir</i>
Real: 5, imaginario: -3i	5.00 - 3.00i
Real: 0, imaginario: 6.27i	6.27i
Real: -8.4, imaginario: 0i	-8.40
Real: 0, imaginario: 0i	0.00
Real: -8.4, imaginario: -2i	-8.40 - 2.00i
Real: 6.1, imaginario: 4i	6.10 + 4.00i

En esta nueva clase para representar números complejos, defina un metodo local, publico, que retorna la *conjugada* del número complejo almacenado en la instancia:

```
public INumero Conjugada( )
```

V. BIBLIOGRAFIA

- Hommel, S.(2007). Early Access Lesson: Learning Swing with the NetBeans IDE. Recuperado de <https://blogs.oracle.com/java/post/early-access-lesson-learning-swing-with-the-netbeans-ide>
- Ruiz, I. (2023). JavaHispano. JTable con Ejemplos-Parte I. Documento web: http://www.javahispano.org/contenidos/archivo/63/jtable_1.pdf
- ORACLE Java Documentation (2025). The Java TM Tutorials. Using Swing Components. Recuperado de <https://docs.oracle.com/javase/tutorial/uiswing/components/table.html>